Universidad Distrital Francisco José de Caldas

Facultad de Ingeniería

Proyecto Curricular de Ingeniería de Sistemas

Documento de Diseño de Solución

NFA Plotter: Traductor de AFN a AFD

Ciencias de la Computación III — Período: 2026-1

Juan Daniel Vanegas Mayorquín

Código: 20222020077

jdvanegasm@udistrital.edu.co

Sebastián Quintero Jaimes

Código: 20221020122

squinteroj@udistrital.edu.co

Docente: Helio Henry Ramirez

Bogotá, Colombia

25 de marzo de 2026

Resumen

El presente documento describe el diseño de solución del proyecto *NFA Plotter*, una aplicación de escritorio desarrollada en Python cuyo propósito es construir un autómata finito no determinista (AFN) a partir de una expresión regular sobre el alfabeto binario $\{0, 1\}$ y traducirlo a un autómata finito determinista (AFD) mediante el método de subconjuntos. El sistema proporciona una interfaz gráfica para validar la expresión ingresada, generar y visualizar el AFN, ejecutar el procedimiento de conversión paso a paso, mostrar la tabla de subconjuntos, renombrar los nuevos estados del AFD y renderizar el autómata resultante.

La solución propuesta utiliza una arquitectura modular orientada a objetos, organizada en capas de dominio, procesamiento de expresiones regulares, algoritmos, visualización e interfaz gráfica. Para la construcción del AFN se implementa desde cero el algoritmo de Thompson, mientras que la conversión AFN a AFD se desarrolla manualmente mediante los procedimientos de cerradura-ε, operación *move*, generación de subconjuntos, identificación de estados de aceptación y eliminación de estados inaccesibles. La interfaz fue construida con PySide6 y la visualización de autómatas se apoya en Graphviz.

El documento expone el alcance del software, los requerimientos funcionales y no funcionales, la arquitectura adoptada, los diagramas de casos de uso y clases, el diseño detallado de los algoritmos, el manejo de errores, la estrategia de pruebas y las decisiones técnicas que justifican la implementación final.

Índice

1. Introducción **5**

2. Planteamiento del problema **5**

3. Objetivos **6**

3.1. Objetivo general 6

3.2. Objetivos específicos 6

4. Alcance del proyecto **7**

5. Requerimientos del sistema **8**

5.1. Requerimientos funcionales 8

5.2. Requerimientos no funcionales 9

6. Decisiones de diseño **9**

6.1. Elección de la forma de entrada 9

6.2. Elección del tipo de aplicación 10

6.3. Uso de Graphviz para visualización 10

6.4. Separación entre lógica y GUI 10

7. Stack tecnológico **10**

8. Arquitectura del sistema **11**

8.1. Módulo `domain` 11

8.2. Módulo `regex` 12

8.3. Módulo `algorithms` 12

8.4. Módulo `visualization` 12

8.5. Módulo `gui` 12

9. Diagramas UML **13**

9.1. Diagrama general de casos de uso 13

9.2. Diagrama de casos de uso del método de subconjuntos 15

9.3. Diagrama de clases 15

10. Diseño orientado a objetos **15**

10.1. Clases del dominio 15

10.2. Clases del procesamiento de expresiones regulares 16

10.3. Clases algorítmicas 16

10.4. Clases de visualización y GUI 16

11. Diseño algorítmico **16**

11.1. Procesamiento de la expresión regular 16

11.2. Construcción del AFN mediante Thompson 17

11.2.1. Caso base: símbolo 17

11.2.2. Concatenación 17

11.2.3. Unión 18

11.2.4. Estrella de Kleene 18

11.2.5. Marcado del AFN final 18

11.3. Método de subconjuntos para AFN a AFD 18

11.3.1. Cerradura-ε 18

11.3.2. Operación *move* 19

11.3.3. Construcción iterativa de subconjuntos 19

11.3.4. Renombrado de estados 20

11.3.5. Estados de aceptación del AFD 20

11.3.6. Estado sumidero 20

11.3.7. Eliminación de estados inaccesibles 20

11.4. Tabla del método de subconjuntos 20

12. Descripción del software **21**

13. Manejo de errores y casos límite **22**

13.1. Expresión vacía 22

13.2. Símbolos inválidos 22

13.3. Paréntesis desbalanceados 23

13.4. Uso inválido de operadores 23

13.5. Subconjunto vacío 23

13.6. Estados inaccesibles 23

13.7. Resultados obsoletos en la GUI 23

14. Estrategia de pruebas **24**

14.1. Pruebas unitarias 24

14.2. Pruebas funcionales manuales 24

14.3. Criterios de aceptación 25

15. Justificación de la solución propuesta **26**

16. Resultados esperados **26**

17. Conclusiones **27**

18. Anexo A. Estructura general del proyecto **27**

1. Introducción

Los autómatas finitos constituyen una de las bases teóricas más importantes de la computación, ya que permiten modelar lenguajes formales, construir analizadores léxicos, describir procesos de reconocimiento y estudiar sistemas de decisión discretos. Dentro de este contexto, la conversión de un AFN a un AFD representa un procedimiento clásico en cursos de lenguajes formales y teoría de autómatas, debido a que demuestra cómo una especificación no determinista puede transformarse en un modelo equivalente y determinista.

En el proyecto desarrollado se propuso construir una aplicación con interfaz gráfica que permitiera al usuario ingresar una especificación formal del autómata, visualizar el AFN generado y posteriormente observar el proceso de traducción a AFD a través del método de subconjuntos. El foco académico del trabajo se centra no solo en obtener un resultado funcional, sino en evidenciar de manera clara y verificable el procedimiento algorítmico involucrado en la construcción del AFN y en la conversión posterior.

Para cumplir con este objetivo se tomó la decisión de representar la entrada del usuario mediante expresiones regulares sobre el alfabeto $\{0, 1\}$, soportando unión, concatenación implícita, estrella de Kleene y agrupación con paréntesis. A partir de esta decisión, el sistema genera primero un AFN usando el algoritmo de Thompson y luego aplica el método de subconjuntos para construir el AFD equivalente. Esta aproximación permitió mantener el proyecto dentro de un alcance razonable, garantizar visualización previa del AFN y conservar el componente algorítmico como núcleo principal de la evaluación.

2. Planteamiento del problema

Se requiere desarrollar un software con interfaz gráfica capaz de ayudar al usuario a comprender y ejecutar la traducción de autómatas finitos no deterministas a autómatas finitos deterministas. El sistema debe permitir ingresar una especificación válida del autómata, representar gráficamente el AFN previo a la conversión, generar la tabla del método de subconjuntos, explicar

los nuevos estados construidos durante el proceso y mostrar finalmente el AFD equivalente.

La dificultad principal del problema radica en que el software no debe limitarse a mostrar el resultado final. También debe exponer el procedimiento intermedio de forma clara, reproducible y comprensible para el usuario. Adicionalmente, el algoritmo de conversión AFN a AFD debía implementarse desde cero, sin delegar la lógica a librerías externas especializadas, ya que el énfasis del curso se encuentra en el análisis y construcción de algoritmos.

A partir de este planteamiento, el proyecto se orientó a resolver tres necesidades centrales: una forma de especificar el lenguaje o autómata de entrada, un mecanismo visual de representación del AFN y una implementación transparente del método de subconjuntos con soporte para tablas, renombrado de estados y visualización final del AFD.

3. Objetivos

3.1. Objetivo general

Diseñar e implementar una aplicación de escritorio en Python que permita construir un AFN a partir de una expresión regular sobre el alfabeto $\{0, 1\}$ y traducirlo a un AFD equivalente mediante el método de subconjuntos, mostrando de forma visual y paso a paso el proceso de conversión.

3.2. Objetivos específicos

1. Definir una arquitectura modular y orientada a objetos que separe el dominio del problema, la lógica algorítmica, la visualización y la interfaz gráfica.

2. Implementar un validador y parser de expresiones regulares con soporte para unión, concatenación implícita, estrella de Kleene y agrupación.

3. Construir un AFN desde cero mediante el algoritmo de Thompson a partir de la expresión regular ingresada por el usuario.

4. Implementar desde cero el método de subconjuntos para traducir el AFN a un AFD equivalente.

5. Visualizar el AFN y el AFD mediante grafos renderizados con Graphviz.
6. Exponer al usuario la tabla de subconjuntos, el renombrado de estados del AFD y los resultados intermedios del proceso.
7. Documentar el diseño del sistema mediante diagramas UML, descripción de clases, casos de uso y estrategia de pruebas.

4. Alcance del proyecto

La solución desarrollada corresponde a una aplicación de escritorio orientada al proceso **AFN → AFD**. Aunque en algunos borradores iniciales apareció la expresión "DFA → NFA translator", el alcance oficial del proyecto es la traducción de un AFN a un AFD usando el método de subconjuntos. Esta definición es coherente con el algoritmo solicitado y con el comportamiento implementado en el software.

El sistema admite expresiones regulares construidas únicamente sobre el alfabeto binario $\{0, 1\}$, con los siguientes operadores:

- unión `|`,
- concatenación implícita,
- estrella de Kleene `*`,
- agrupación mediante paréntesis.

La aplicación no implementa dibujo manual del autómata. En lugar de ello, se optó por un único mecanismo de entrada: la especificación textual mediante expresión regular. Esta decisión se justificó por tres razones: reduce el tiempo de desarrollo, concentra el esfuerzo en el componente algorítmico exigido por la materia y mantiene el cumplimiento del requisito de visualización, ya que el AFN generado puede observarse gráficamente antes de iniciar la conversión.

El proyecto tampoco contempla persistencia de datos, autenticación, soporte multiusuario ni minimización formal del AFD. El producto se enfoca en la construcción, explicación y visualización del proceso de traducción.

5. Requerimientos del sistema

5.1. Requerimientos funcionales

ID	Descripción
RF-01	El sistema debe permitir al usuario ingresar una expresión regular sobre el alfabeto $\{0, 1\}$.
RF-02	El sistema debe validar sintácticamente la expresión regular ingresada.
RF-03	El sistema debe transformar la expresión regular válida a una representación intermedia adecuada para su procesamiento.
RF-04	El sistema debe construir un AFN a partir de la expresión regular usando el algoritmo de Thompson.
RF-05	El sistema debe mostrar gráficamente el AFN generado antes de ejecutar la traducción a AFD.
RF-06	El sistema debe aplicar el método de subconjuntos para traducir el AFN a un AFD equivalente.
RF-07	El sistema debe mostrar la tabla paso a paso del método de subconjuntos, incluyendo subconjuntos, operación *move*, cerradura-ε y estados destino.
RF-08	El sistema debe renombrar los subconjuntos construidos con identificadores del tipo $K_0, K_1, K_2, \ldots$.
RF-09	El sistema debe identificar correctamente los estados de aceptación del AFD.
RF-10	El sistema debe eliminar del AFD final los estados inaccesibles.
RF-11	El sistema debe renderizar y mostrar visualmente el AFD generado.

RF-12	El sistema debe mostrar mensajes de error cuando la expresión regular sea inválida.

5.2. Requerimientos no funcionales

ID	Descripción
RNF-01	La aplicación debe ejecutarse como software de escritorio.
RNF-02	La solución debe implementarse en Python.
RNF-03	La lógica de conversión AFN a AFD debe implementarse desde cero, sin depender de librerías que resuelvan el algoritmo.
RNF-04	El código debe organizarse de forma modular y orientada a objetos para facilitar mantenimiento, pruebas y documentación.
RNF-05	La interfaz gráfica debe ser lo suficientemente clara para que el usuario pueda seguir el proceso completo de validación, construcción y conversión.
RNF-06	El proyecto debe incluir documentación técnica, diagramas UML y pruebas de funcionamiento.
RNF-07	El entorno del proyecto debe gestionarse mediante Poetry.

6. Decisiones de diseño

6.1. Elección de la forma de entrada

La primera decisión importante del proyecto fue elegir la forma en que el usuario plantearía el autómata. Se consideraron dos opciones: permitir dibujo manual del AFN o permitir especificación textual mediante expresión regular. Finalmente se eligió únicamente la segunda alternativa.

La entrada textual ofrece una implementación más controlada y reduce considerablemente la complejidad de la interfaz. Además, permite concentrar el esfuerzo en los algoritmos, que son el componente central de la evaluación. Esta decisión no afecta el cumplimiento del requerimiento de

visualización, porque el sistema genera el AFN y lo renderiza antes de la conversión.

6.2. Elección del tipo de aplicación

Se optó por una aplicación de escritorio desarrollada totalmente en Python. Esta elección evita tener que dividir la solución en frontend y backend, elimina la necesidad de desplegar una API y simplifica la arquitectura. También facilita el uso de un diseño orientado a objetos y la elaboración del diagrama de clases.

6.3. Uso de Graphviz para visualización

La generación de grafos se resolvió mediante Graphviz, ya que permite construir de manera sencilla representaciones DOT y exportarlas a imágenes PNG. Esto reduce el esfuerzo de desarrollo en la parte gráfica, sin afectar la lógica algorítmica que sí fue implementada manualmente.

6.4. Separación entre lógica y GUI

La interfaz gráfica se diseñó como una capa de orquestación. Su función principal consiste en recibir la entrada del usuario, invocar la validación, mostrar resultados, renderizar imágenes y actualizar tablas. Toda la lógica formal del procesamiento de expresiones regulares, construcción del AFN, conversión AFN a AFD y poda de estados inaccesibles reside en módulos separados. Esta separación facilita pruebas unitarias, mantenimiento y trazabilidad.

7. Stack tecnológico

La Tabla 3 resume las tecnologías empleadas en el proyecto y su propósito principal.

Tabla 3: Stack tecnológico del proyecto

Tecnología	Versión objetivo	Propósito
Python	3.11 – 3.14	Lenguaje principal de desarrollo.
Poetry	2.3.2	Gestión de dependencias, entorno y scripts del proyecto.
PySide6	6.11.x	Construcción de la interfaz gráfica desktop.
Graphviz (sistema)	2.43.0	Motor de renderizado de grafos.
graphviz (Python)	0.21	Generación de código DOT y exportación de imágenes.
pytest	9.x	Pruebas unitarias y validación del comportamiento del software.
pdoc	16.x	Generación de documentación basada en docstrings.
Ruff / Black / mypy	varias	Calidad estática del código, formato y chequeo básico de tipos.

La elección de este stack responde a criterios de uniformidad, velocidad de desarrollo y claridad arquitectónica. Mantener toda la solución en Python permitió reducir fricción técnica y facilitar la integración entre la GUI, la lógica de negocio, los algoritmos y las pruebas.

8. Arquitectura del sistema

El sistema se organizó en cinco módulos principales: `domain`, `regex`, `algorithms`, `visualization` y `gui`. Esta división persigue una arquitectura modular donde cada parte tenga responsabilidades específicas y de bajo acoplamiento.

8.1. Módulo `domain`

Contiene las abstracciones fundamentales del problema: estados, transiciones y autómatas. Aquí se encuentran las clases `State`, `Transition`, `Automaton`, `NFA` y `DFA`. Este módulo define la

estructura del modelo con la que trabajan los demás componentes.

8.2. Módulo `regex`

Se encarga del procesamiento de expresiones regulares. Incluye validación de entrada, inserción explícita de concatenación, conversión a notación postfix y construcción del AFN con Thompson. Su objetivo es transformar una cadena de entrada en una estructura formal manipulable por el sistema.

8.3. Módulo `algorithms`

Contiene el núcleo algorítmico del proyecto. Allí se implementan la cerradura-ε, la operación *move*, el convertidor por método de subconjuntos y el módulo de poda de estados inaccesibles. Este módulo es el corazón académico de la solución.

8.4. Módulo `visualization`

Se encarga de convertir autómatas y resultados de conversión en salidas visuales o tabulares. Incluye el renderizador de autómatas con Graphviz y el formateador de la tabla de subconjuntos.

8.5. Módulo `gui`

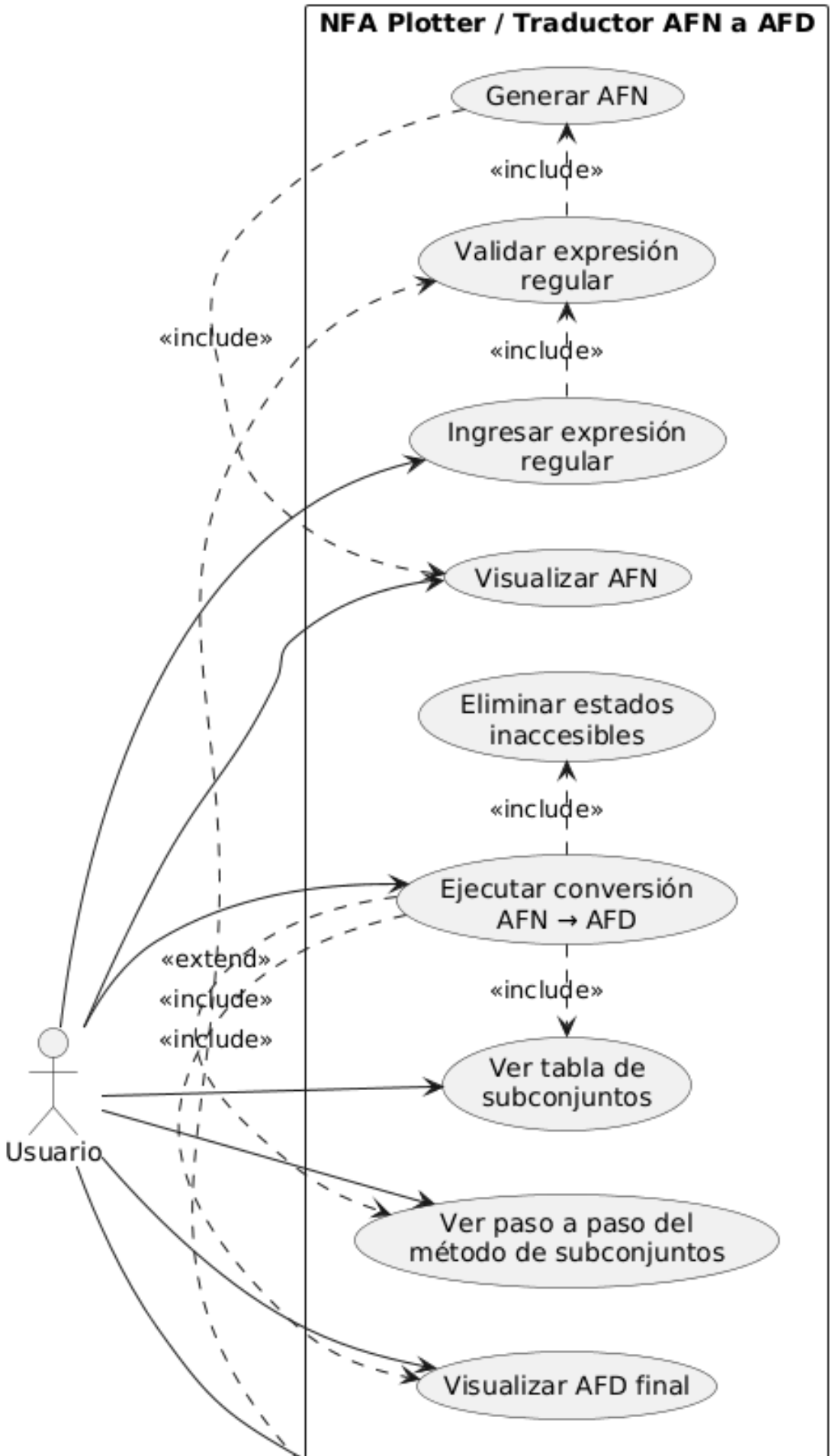
Gestiona la interacción con el usuario a través de una ventana principal que integra:

- entrada de expresiones regulares,
- visualización del AFN,
- visualización del AFD,
- tabla del método de subconjuntos,
- mensajes de error y estado,
- resúmenes textuales de estructuras y mapeos.

9. Diagramas UML

9.1. Diagrama general de casos de uso

La Figura 1 muestra el comportamiento global del sistema desde la perspectiva del usuario. Se identifican las acciones de ingresar una expresión regular, validarla, generar el AFN, visualizarlo, ejecutar la conversión AFN a AFD, consultar la tabla de subconjuntos y observar el AFD final.


NFA Plotter / Traductor AFN a AFD
Generar AFN
«include»
Validar expresión regular
«include»
Ingresar expresión regular
«include»
Visualizar AFN
Eliminar estados inaccesibles
«include»
Ejecutar conversión AFN → AFD
«extend»
«include»
«include»
«include»
Ver tabla de subconjuntos
Usuario
Ver paso a paso del método de subconjuntos
Visualizar AFD final

9.2. Diagrama de casos de uso del método de subconjuntos

La Figura 2 detalla específicamente el flujo de conversión AFN a AFD. Allí se explicita que el sistema calcula la cerradura-ε inicial, construye subconjuntos, evalúa la operación *move*, crea nuevos estados del AFD, los renombra, identifica aceptación y genera la tabla y el autómata final.

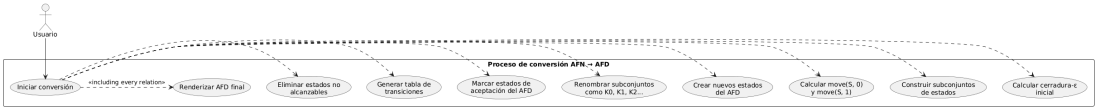


Figura 2: Diagrama de casos de uso enfocado en el método de subconjuntos.

9.3. Diagrama de clases

La Figura 3 presenta la estructura orientada a objetos de la solución. El diseño separa claramente los modelos del dominio, los servicios de procesamiento de expresiones regulares, los algoritmos de conversión, la capa de visualización y la capa de interfaz gráfica.

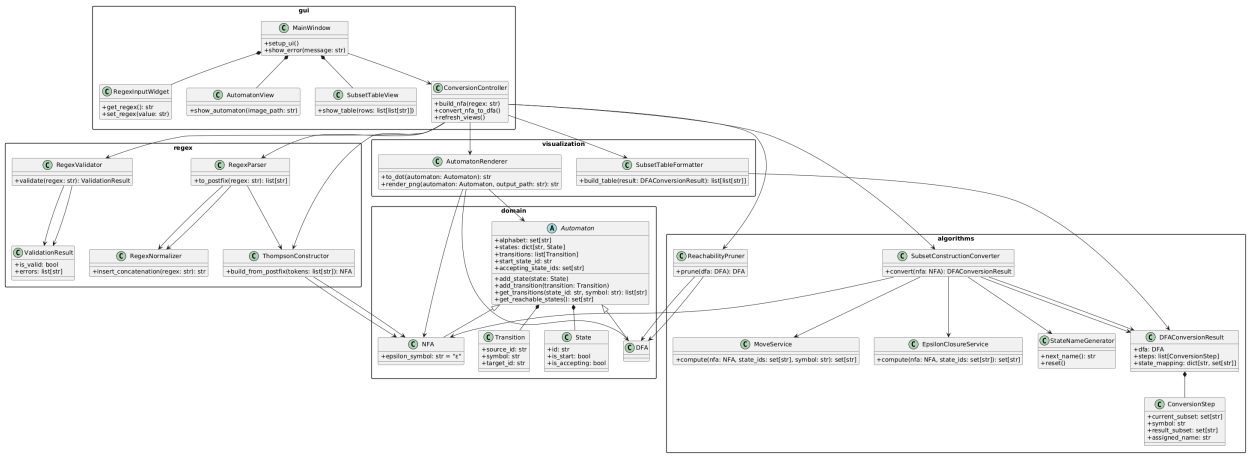


Figura 3: Diagrama de clases del proyecto.

10. Diseño orientado a objetos

10.1. Clases del dominio

La clase `State` representa un estado individual del autómata y almacena su identificador, así como marcas de inicio y aceptación. La clase `Transition` modela una transición entre dos estados

etiquetada por un símbolo. La clase abstracta `Automaton` define la estructura base compartida por AFN y AFD: alfabeto, conjunto de estados, transiciones, estado inicial y estados de aceptación. A partir de ella se derivan `NFA` y `DFA`, cada una con reglas particulares de validación.

10.2. Clases del procesamiento de expresiones regulares

La clase `RegexValidator` verifica la validez sintáctica de la expresión regular. `RegexNormalizer` inserta explícitamente el operador de concatenación interna, facilitando el trabajo del parser. `RegexParser` transforma la expresión de notación infija a notación postfix. Finalmente, `ThompsonConstructor` toma la expresión postfix y genera el AFN equivalente.

10.3. Clases algorítmicas

`EpsilonClosureService` calcula la cerradura-ε de un conjunto de estados. `MoveService` obtiene los estados alcanzables desde un subconjunto con un símbolo dado. `SubsetConstructionConverter` coordina la conversión AFN a AFD, conserva la tabla de pasos y el mapeo de subconjuntos a estados renombrados. `ReachabilityPruner` elimina estados inaccesibles del AFD resultante.

10.4. Clases de visualización y GUI

`AutomatonRenderer` transforma autómatas en grafos DOT y los renderiza como imágenes. `SubsetTableFormatter` construye las filas legibles de la tabla del método de subconjuntos. La clase `MainWindow` organiza toda la interacción visual, integra los servicios y actualiza la presentación de resultados.

11. Diseño algorítmico

11.1. Procesamiento de la expresión regular

El flujo algorítmico del sistema comienza con el tratamiento de la expresión regular. Antes de construir cualquier autómata, la entrada pasa por tres etapas:

1. **Validación sintáctica:** se verifica que la cadena no esté vacía, que solo contenga símbolos válidos, que los paréntesis estén balanceados y que los operadores estén bien ubicados.

2. **Normalización:** se inserta explícitamente el operador de concatenación donde sea necesario. Por ejemplo, la expresión `(0|010)*` se transforma a `(0|0.1.0)*`.

3. **Conversión a postfix:** se reordena la expresión utilizando un parser tipo *shunting-yard*, facilitando la construcción posterior del AFN.

La forma postfix evita ambigüedades de precedencia entre unión, concatenación y estrella de Kleene, y permite procesar la expresión mediante una pila de fragmentos.

11.2. Construcción del AFN mediante Thompson

Una vez obtenida la expresión en postfix, el sistema aplica el algoritmo de Thompson. Este método construye el AFN a partir de fragmentos parciales que se combinan según el operador encontrado.

11.2.1. Caso base: símbolo

Para cada símbolo del alfabeto, se crea un fragmento elemental con dos estados y una transición etiquetada con dicho símbolo. Por ejemplo, para el símbolo `0`, se generan dos estados q_i y q_j con la transición:

$$\delta(q_i, 0) = q_j$$

11.2.2. Concatenación

Si el token actual es el operador de concatenación, se extraen dos fragmentos de la pila: uno izquierdo y uno derecho. Luego se conecta el estado de aceptación del fragmento izquierdo con el estado inicial del fragmento derecho mediante una transición ε. El nuevo fragmento tiene como estado inicial el del fragmento izquierdo y como estado de aceptación el del fragmento derecho.

11.2.3. Unión

Si el token corresponde a la unión, se crean un nuevo estado inicial y un nuevo estado de aceptación. Desde el nuevo inicio se añaden dos transiciones ε hacia los inicios de los dos fragmentos operandos. De igual forma, desde los estados de aceptación de ambos fragmentos se agregan transiciones ε hacia el nuevo estado final.

11.2.4. Estrella de Kleene

Para la estrella de Kleene se crea también un nuevo estado inicial y un nuevo estado de aceptación. Desde el nuevo inicio se generan transiciones ε hacia el inicio del fragmento original y hacia el nuevo final. Desde el estado de aceptación del fragmento original se agregan transiciones ε hacia su propio inicio y hacia el nuevo final. Esto permite aceptar cero o más repeticiones del patrón.

11.2.5. Marcado del AFN final

Cuando toda la expresión postfix ha sido procesada, debe quedar un único fragmento en la pila. El estado inicial y el estado de aceptación de dicho fragmento se marcan como inicio y aceptación del AFN completo.

11.3. Método de subconjuntos para AFN a AFD

El método de subconjuntos constituye el núcleo de la traducción implementada. Su principio central consiste en que cada estado del AFD representa un subconjunto de estados del AFN original.

11.3.1. Cerradura-ε

La cerradura-ε de un conjunto de estados se define como el conjunto de todos los estados alcanzables desde dichos estados siguiendo cero o más transiciones ε. En la implementación, este cálculo se realiza mediante un recorrido usando una cola, partiendo de los estados iniciales y expandiendo todos los destinos alcanzables con ε.

Si el AFN tiene estado inicial q_0, entonces el primer estado del AFD no es simplemente $\{q_0\}$, sino:

$$\varepsilon\text{-closure}(\{q_0\})$$

Este conjunto se convierte en el primer estado del AFD y se renombra como K_0.

11.3.2. Operación move

La operación *move* recibe un conjunto de estados del AFN y un símbolo del alfabeto. Su resultado es el conjunto de todos los estados alcanzables desde cualquiera de esos estados utilizando exactamente ese símbolo. Formalmente:

$$move(S, a) = \bigcup_{q \in S} \delta(q, a)$$

En el proyecto, `MoveService` implementa esta operación reuniendo todos los destinos alcanzables para un símbolo dado.

11.3.3. Construcción iterativa de subconjuntos

A partir del subconjunto inicial, el algoritmo procesa cada estado del AFD pendiente. Para cada símbolo del alfabeto se hace lo siguiente:

a) Se toma el subconjunto actual S.

b) Se calcula $move(S, a)$.

c) Se aplica cerradura-ε al resultado anterior.

d) El conjunto obtenido representa el estado destino del AFD con símbolo a.

Si ese subconjunto no había sido visto antes, se crea un nuevo estado del AFD, se le asigna un nombre del tipo K_i y se agrega a la cola de procesamiento. Si ya existía, simplemente se reutiliza su identificador.

11.3.4. Renombrado de estados

Para que el resultado sea legible y didáctico, cada subconjunto del AFN se mapea a un nombre secuencial del tipo $K_0, K_1, K_2, \ldots$. Esto separa dos niveles de representación:

- la representación formal interna como subconjunto del AFN, por ejemplo $\{q_0, q_2, q_5\}$,
- la representación visual del estado del AFD, por ejemplo K_3.

El sistema conserva este mapeo y lo muestra al usuario en un panel de texto adicional.

11.3.5. Estados de aceptación del AFD

Un estado del AFD se marca como de aceptación si el subconjunto del AFN que representa contiene al menos un estado de aceptación del AFN original. Esta regla se aplica durante la construcción de los estados del AFD.

11.3.6. Estado sumidero

Cuando para un símbolo dado la operación *move* y la cerradura-ε resultan en el conjunto vacío, el algoritmo crea un estado asociado a $\emptyset$. Este comportamiento es correcto, pues en el AFD dicho estado actúa como sumidero alcanzable cuando una transición no conduce a ningún estado útil del AFN.

11.3.7. Eliminación de estados inaccesibles

Una vez construido el AFD, se ejecuta una poda final para eliminar estados que no sean alcanzables desde el estado inicial. Esta fase no elimina el estado sumidero si este es alcanzable. Solo remueve estados realmente desconectados del recorrido del autómata.

11.4. Tabla del método de subconjuntos

Durante la conversión, el sistema registra cada paso con la siguiente información:

- nombre del estado fuente del AFD,
- subconjunto del AFN que representa,
- símbolo procesado,
- resultado de la operación *move*,
- resultado de la cerradura-ε,
- nombre del estado destino,
- indicador de si el estado destino es nuevo,
- indicador de si es un estado de aceptación.

Esta tabla cumple una función didáctica importante, porque permite seguir con precisión cómo se van generando y reutilizando los estados del AFD.

12. Descripción del software

La interfaz del sistema fue diseñada para que el usuario recorra el proceso completo en una sola ventana. El flujo de uso es el siguiente:

1. El usuario ingresa una expresión regular.
2. Puede validarla y observar su forma normalizada y postfix.
3. El usuario genera el AFN y lo visualiza.
4. Posteriormente ejecuta la conversión AFN a AFD.
5. El sistema muestra el AFD, la tabla del método de subconjuntos y el mapeo de estados renombrados.

La ventana principal integra varias áreas de información:

- campo de entrada de la expresión regular,
- botones para validar, generar AFN, convertir a AFD y limpiar,
- panel de estado y mensajes,
- resumen textual del AFN,
- resumen textual del AFD,
- panel de mapeo de estados del AFD,
- tabla del método de subconjuntos,
- visualización gráfica del AFN,
- visualización gráfica del AFD.

Una mejora importante incorporada durante el desarrollo fue la limpieza automática de resultados previos cuando el usuario modifica la expresión regular. Esto evita inconsistencias visuales y garantiza que la interfaz no muestre resultados obsoletos.

13. Manejo de errores y casos límite

Durante el diseño se contemplaron múltiples casos que podían comprometer la validez del procesamiento o la claridad de la interfaz. Entre ellos se destacan los siguientes:

13.1. Expresión vacía

Si el usuario no ingresa ningún contenido, el sistema rechaza la entrada y emite un mensaje indicando que la expresión regular no puede estar vacía.

13.2. Símbolos inválidos

Solo se admiten los símbolos `0`, `1`, los operadores `|` y `*`, así como los paréntesis. Cualquier carácter adicional genera error de validación.

13.3. Paréntesis desbalanceados

El sistema detecta cierres inesperados, grupos incompletos o balance incorrecto de paréntesis y no permite avanzar hacia la construcción del AFN.

13.4. Uso inválido de operadores

Se contemplan errores como:

- inicio con operador inválido,
- uniones duplicadas,
- aplicación consecutiva incorrecta de estrella,
- grupos vacíos,
- expresiones terminadas con operadores incompletos.

13.5. Subconjunto vacío

Durante la conversión AFN a AFD puede aparecer el subconjunto vacío. Este no se considera un error, sino un caso formalmente válido del método de subconjuntos, y se representa como estado sumidero.

13.6. Estados inaccesibles

Después de la conversión, cualquier estado que no sea alcanzable desde el estado inicial se elimina del AFD final para mantener una representación más limpia y coherente.

13.7. Resultados obsoletos en la GUI

Si el usuario cambia la expresión regular después de haber generado un AFN o un AFD, el sistema limpia automáticamente las salidas previas para impedir interpretaciones erróneas.

14. Estrategia de pruebas

La validación del proyecto se abordó mediante pruebas unitarias y pruebas manuales.

14.1. Pruebas unitarias

Las pruebas unitarias verifican comportamientos específicos de los módulos implementados:

- aceptación y rechazo de expresiones regulares en el validador,
- inserción correcta del operador de concatenación,
- conversión a postfix,
- construcción de AFN con Thompson,
- cálculo de cerradura-ε,
- operación *move*,
- construcción correcta del AFD,
- identificación de estados de aceptación,
- aparición del estado sumidero cuando corresponde,
- poda de estados inaccesibles.

Estas pruebas se ejecutan mediante `pytest` y permiten detectar fallos lógicos de forma temprana.

14.2. Pruebas funcionales manuales

Se definieron casos de prueba manuales orientados al uso real de la interfaz. Algunos ejemplos son:

Tabla 4: Casos de prueba funcionales representativos

Entrada	Resultado esperado
`0`	Expresión válida, AFN de dos estados, AFD con estado sumidero y tabla de conversión visible.
`01`	Validación correcta, normalización a `0.1`, AFN secuencial, AFD generado sin errores.
`0|1`	AFN con transiciones ε de unión, AFD con al menos un estado de aceptación.
`(0|1)*`	AFN con ciclos ε, AFD válido y estado inicial de aceptación.
`(0|010)*`	Construcción correcta del AFN, tabla del método de subconjuntos, mapeo de estados K_i y visualización del AFD.
`|0`	Error de validación y bloqueo del proceso de construcción.
`()`	Error por paréntesis vacíos, sin generar AFN ni AFD.
`0||1`	Error de validación, limpieza de resultados previos y mensaje claro al usuario.

14.3. Criterios de aceptación

Se consideró que el software cumple adecuadamente su propósito cuando:

1. las expresiones válidas generan AFN y AFD sin excepciones inesperadas;
2. las expresiones inválidas muestran mensajes comprensibles;
3. la tabla del método de subconjuntos coincide con el comportamiento del AFD generado;
4. los estados de aceptación del AFD están correctamente identificados;
5. la interfaz mantiene consistencia visual durante el uso.

15. Justificación de la solución propuesta

La solución planteada es adecuada para el problema por varias razones. En primer lugar, satisface el requerimiento de contar con interfaz gráfica y de permitir la visualización del AFN antes de la traducción. En segundo lugar, mantiene el énfasis del proyecto en la parte algorítmica, dado que tanto Thompson como el método de subconjuntos fueron implementados desde cero. En tercer lugar, la arquitectura modular facilita que cada parte del sistema pueda estudiarse, probarse y documentarse de manera independiente.

Otro aspecto relevante es que la elección de entrada por expresión regular no reduce el valor académico del proyecto. Por el contrario, añade una etapa formal adicional al flujo del sistema, porque exige validar, normalizar y parsear la expresión antes de construir el AFN. Esto enriquece el proyecto y lo conecta directamente con la teoría de lenguajes formales.

Finalmente, la inclusión de tablas, mapeos de estados y visualizaciones gráficas transforma el software en una herramienta útil no solo para ejecutar la conversión, sino también para comprenderla y explicarla.

16. Resultados esperados

Como producto final del proyecto se espera disponer de una aplicación que:

- reciba expresiones regulares válidas sobre el alfabeto $\{0, 1\}$;
- construya correctamente el AFN equivalente;
- muestre el AFN antes de la traducción;
- convierta el AFN a AFD usando el método de subconjuntos;
- produzca una tabla clara con los pasos relevantes del proceso;
- muestre el mapeo entre subconjuntos del AFN y estados del AFD;
- renderice correctamente el AFD final.

Desde el punto de vista académico, el resultado esperado es un software que permita evidenciar de forma clara la aplicación práctica de conceptos de teoría de autómatas, estructuras de datos, recorridos sobre grafos, procesamiento de expresiones regulares y diseño modular de software.

17. Conclusiones

El diseño de solución desarrollado para *NFA Plotter* demuestra que es posible integrar de manera coherente teoría de autómatas, algoritmos formales y desarrollo de software con interfaz gráfica en una aplicación única, clara y funcional. La decisión de utilizar expresiones regulares como mecanismo de entrada permitió reducir complejidad innecesaria en la interfaz y concentrar el trabajo en los componentes algorítmicos de mayor valor académico.

La implementación modular en Python facilitó separar responsabilidades y construir un sistema mantenible. El uso de clases especializadas para validación, parseo, construcción de AFN, conversión a AFD, renderizado y gestión de interfaz permitió alinear la solución con buenas prácticas de diseño orientado a objetos y respaldar de forma consistente el diagrama de clases propuesto.

Desde la perspectiva algorítmica, la construcción del AFN mediante Thompson y la traducción AFN a AFD por método de subconjuntos constituyen el núcleo técnico del proyecto. La implementación de cerradura-ε, *move*, generación de subconjuntos, renombrado de estados, identificación de aceptación y poda de estados inaccesibles cumple con el alcance establecido y con la exigencia de desarrollar la lógica desde cero.

En términos funcionales, el sistema no solo produce autómatas correctos, sino que además explica el proceso mediante resúmenes, tablas y visualizaciones. Esto convierte la solución en una herramienta útil tanto para la validación del proyecto como para el apoyo en el aprendizaje de los conceptos involucrados.

18. Anexo A. Estructura general del proyecto

La estructura general del repositorio implementado se organizó de la siguiente manera:

```
src/nfa_plotter/
 algorithms/
    epsilon_closure.py
    move.py
    reachability.py
    subset_construction.py
    __init__.py
 domain/
    state.py
    transition.py
    automaton.py
    nfa.py
    dfa.py
    __init__.py
 gui/
    main_window.py
    __init__.py
 regex/
    validation.py
    normalization.py
    parser.py
    thompson.py
    __init__.py
 visualization/
    renderer.py
    subset_table.py
    __init__.py
```

`main.py`

`__main__.py`

Esta organización refleja directamente la arquitectura explicada en el presente documento y facilita la trazabilidad entre diseño, implementación y pruebas.